

# 72.1 → 87.5

The original queue of 90 items is done, the 1,730-line Home screen is split, and ten real bugs turned up along the way — most of them things a user would have noticed. A fresh review of the code that now ships raised 85 more findings; twelve pieces of work closed 25 of them, and the remaining 60 are listed rather than swept. One change still needs checking on a real phone before you ship it.

|                                                                                                           |                                                                                                               |                                                                                                     |                                                                                              |
|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <div>STRICT SCORE</div> <div><del>72.1</del> → <b>87.5</b></div> <div>past the tool's own 85 target</div> | <div>MECHANICAL SCORE</div> <div><del>79.7</del> → <b>96.2</b></div> <div>file sizes, duplication, lint</div> | <div>ORIGINAL QUEUE</div> <div><del>90</del> → <b>0</b></div> <div>then 25 of 85 new findings</div> | <div>APP TESTS</div> <div><del>282</del> → <b>514</b></div> <div>plus backend, 55 → 70</div> |
|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|

## BUGS FOUND WHILE CLEANING

These weren't on the list. They surfaced because splitting a file forces you to read it properly. Each one is described by what you'd have seen, not by what the code did.

USERS SAW IT

### Every comment showed the wrong time

Comment timestamps were saved in UTC but read back as if they were local time, so every comment was off by your timezone offset — three hours out in East Africa, eight in California. Timestamps are now stored the same way as bookmark and download dates.

USERS SAW IT

### The AI comparison feature was switched off in every correct build

Bookmarks refused to generate a comparison unless an OpenAI key was present in the app. That key had already been moved to the server for security — so in any properly configured build the variable is empty and the feature simply declined to run. The check is gone; the server owns that decision.

---

USERS SAW IT

## One phone could like the same solution over and over

On the Bookmarks screen, every tap of the thumbs-up added another like, with no record that you'd already liked it. Everywhere else in the app it toggles. All four screens now share one implementation, so the count can't be run up.

---

USERS SAW IT

## Unknown cost and complexity were labelled "Moderate"

Three different places mapped an unrecognised value to the middle of the scale, so a guess was presented as a specific answer — including in the documents users export and keep. The drawer now shows a dash, and exports say "Not specified".

---

USERS SAW IT

## Comparison summaries could stop mid-word

The AI was given a hard length limit but only *asked* to be brief, so a long reply got sliced wherever the budget ran out and the fragment was shown as written. It now trims back to the last finished sentence.

---

USERS SAW IT

## Reduced motion was ignored while the app started

The setting is read from storage, and until it arrived the app assumed motion was fine — during the launch sequence, which is when it runs the most animation at once. Someone who had asked for reduced motion got the animations anyway. It now stays still until it knows otherwise.

---

COULD HAVE HUNG

## A search with no results could spin forever

When a search finds nothing, the app fetches hotlines to offer instead. If the server ever reported "more results available" while returning an empty page, that fetch looped indefinitely, asking for the same page over and over. An empty page now ends the loop regardless of what the flag says.

---

QUIET DATA LOSS

## Liking something while more results loaded undid the like

The "load more" handler held a stale copy of the results list, so appending the next page reverted any like registered in between.

---

#### QUIET DATA LOSS

### A summary section could be swallowed by the one above it

The comparison summary is split into Use Case, Approach and Complexity/Cost. If one heading came directly after another with no blank line, the second was never recognised and its text was folded into the previous section. Two more spellings the AI commonly uses were also unrecognised and rendered as raw capitals.

---

#### SPEED

### Opening a category made 152 database calls instead of 7

Loading a list of 30 solutions issued five separate database queries for every single row. It now fetches everything in seven queries no matter how many rows come back.

---

#### THE SECOND PASS

---

The fresh review raised 85 findings. Twelve pieces of work were picked, chosen on one rule: a user could be harmed by it, or it costs almost nothing. Between them they close 25 findings — several describe the same defect from a different angle. Eight were real bugs. Each is described by what you'd have seen.

#### USERS SAW IT

### Filters silently returned the wrong solutions

A filter value containing % or \_ was handed to the database as a wildcard instead of as text — so \_ quietly matched *any* character in its place and returned solutions that don't belong in that slice. The fix already existed in the codebase; it had been wired into two of the ten places that needed it. All ten now use it. Nothing was ever injectable, and no current category name contains those characters, so this was a trap waiting on the next piece of data rather than a live fault.

---

#### USERS SAW IT

### Dictation could stick on "transcribing" forever

Every other call to the server gives up after a set time. The one that uploads your recording had no limit at all — so a stalled connection left the microphone in the transcribing state permanently, with no error and no way to retry short of restarting the app. It now gives up after sixty seconds, which is generous because it's uploading a file, and tells you it timed out.

---

USERS SAW IT

## Both heat maps showed a spinner that never stopped

If either heat map failed to load, the failure was written to the developer console and nothing else happened — and "no data yet" is exactly how both of them draw a load still in progress. So a failure looked identical to a slow load, forever. Both now say the load failed and offer a retry that genuinely retries.

---

USERS SAW IT

## An empty search could leave the help section spinning

Search returns nothing → the app starts fetching hotlines → you search again before it finishes → that second search is also empty. The interrupted fetch never cleared its loading flag and never released the "only fetch once" latch, so the help section sat spinning and could never try again for the rest of that visit.

---

USERS SAW IT

## The results list showed "Loading more..." when it wasn't

The footer spinner was wired to "there are more pages" rather than to "a page is loading right now", so it was pinned on for the entire time you scrolled a multi-page result. The correct value was already being calculated and simply never read.

---

WAITING TO BREAK

## Two database modules imported each other

The only circular import in the app, and it existed for one three-line helper that had been left behind in the wrong file — the file that owns it never even used it. It works today by accident of load order. Moved to its own small module, with tests.

---

SAID ONE THING, DID ANOTHER

## A counting function promised a safety limit it never applied

Its documentation said a broad filter would stop counting at a ceiling rather than scan the whole table. No ceiling was ever passed. Here the code was right and the description was wrong: the catalogue is a fixed ~3,000 rows, and truncating the total would make the list stop paging early and hide real results. So the description now says what actually happens and why, and the truncation signal it used to throw away is read and logged.

---

#### DEAD CODE

### A billing guard that could never fire

Search carried a documented protection against re-running an identical query and re-billing the AI call. Both callers overrode it, and the blur-and-refocus events it was written for never call search at all. It also fought the two things that *should* re-run: submitting the same text again is how you retry after a failure. Removed, rather than left looking like a live safeguard.

---

#### HOUSEKEEPING

### Half the repository was a generated test report

`coverage/` — 104 of 221 tracked files, 2.8 MB of generated HTML — was committed, wasn't ignored, and described a folder layout the refactor had already removed. Every coverage run dirtied the working tree. Now ignored and removed from tracking.

---

#### HOUSEKEEPING

### A setup step nobody could have followed

The backend can require a shared token on every request. The app sends one. It appeared in no example file and no instructions — so anyone who set the token on the server and followed the setup guide got a blanket "unauthorized" on search, dictation and both summary features, with nothing anywhere to explain it. Now documented on both sides, with a troubleshooting entry.

---

#### HOUSEKEEPING

### The README described a layout that no longer exists

It pointed four times at a file the refactor deleted, and had no entry at all for three of the folders the app is now organised around. The architecture document was already correct, so the README was brought in line with it.

---

#### TIDYING

### One block of wiring, written out three times

The twelve lines that connect a solution card to the like/bookmark/download behaviour were byte-for-byte identical in all three Home views, so adding a button to a card meant editing three files and hoping they stayed in step. Now one component.

---

## THE BIG SPLIT

---

Four files were doing several jobs each. Nothing about how the app behaves changed — the point was that these parts could not be tested or understood separately.

**HomeScreen.js** 1,730 lines, 45 pieces of state → 263

```
HomeScreen.js      the shell: which half you're on, and what spans both
  home/SearchMode.js  the landing page and the results list
  home/ExploreMode.js stats, category grids, hubs, recent solutions
  home/DrilldownView.js a filtered slice of the catalogue
+ five hooks holding search, explore data, drilldown, help lookup
  and the shared bookmark/like/download behaviour
```

**db.js** 1,040 lines → 355, as five modules

```
connection.js  opening the database, and the tables this app adds
db.js          the read-only catalogue queries
enrich.js      turning a raw row into a solution
engagement.js  likes, comments, cached summaries – the only writes
heatmaps.js    the two grids
```

**BookmarksScreen.js** 585 → 215 · **FilterPanel.js** 709 → 492

The 350-line comparison feature moved out of Bookmarks, where it had nothing to do with listing bookmarks. In the filter panel, three separate patterns had each been written out two to four times — the category pickers, the readiness and adoption sliders, and the toggle-chip grids. Several of those copies had already drifted apart from each other.

## THE TESTS THAT WERE IMPOSSIBLE BEFORE

---

The previous session wrote six tests that drive a search from typing to results, watched them fail, and deleted them — the submitting button only appeared after an interaction locked inside the screen's own state, so no test could reach it. That was the argument for splitting the file.

Those tests are back and passing. A search now runs end to end in the suite: typed query, backend call, ordered results, error, retry, and the empty-result hotline state. The suite went from 282 to 499 in that pass and stands at 514 after the second one, and the new tests are mostly of things that were previously only assumed — including one that pins the filter contract across the three separate modules that each interpret it, which nothing had checked before.

## WHAT THE SCORE ACTUALLY SAYS

---

Both halves of the score were recalculated. The mechanical half is counted by the tool; the other twenty dimensions are judgement calls, so they were re-graded by twenty independent reviewers — one per

dimension, each told to assess the code on its current merits rather than approve a recent refactor.

| SCORE      | BEFORE | NOW  | BASIS                     |
|------------|--------|------|---------------------------|
| Strict     | 72.1   | 87.5 | re-graded · target was 85 |
| Overall    | 75.3   | 88.3 | re-graded                 |
| Mechanical | 79.7   | 96.2 | measured                  |

Every one of the twenty judgement ratings went up. The three that had been dragging hardest moved most, and they are exactly the three this work targeted: incomplete migration 66.5 → **91.5**, authorization consistency 69.5 → **83.0**, test strategy 64.0 → **76.0**. High-level elegance — the single biggest drag on the old number — went 72.5 → **88.0**.

Test strategy is still the lowest at 76.0, which is fair: the pure modules are thoroughly covered, the screens less so. That is the honest next piece of work.

### Why the headline number didn't move for the second pass

Only the mechanical score is measured. It went 96.1 → **96.2**, which is all the twelve fixes could move on their own: most of them were correctness, and correctness is not something a file-size-and-duplication counter can see.

The other three quarters of the score are the twenty judgement ratings, and those are *stale* — ten of them are now explicitly flagged as assessed against code that has since changed. Re-grading them means running twenty fresh independent reviews, which is a deliberate exercise and not something to fold into a cleanup pass, so **87.5 is last round's judgement, carried forward unchanged**. It is not a claim that these twelve fixes were worth nothing.

The honest reading: the score is a fair picture of the code as of the first pass, and slightly pessimistic about where it stands now. The right time to re-grade is after the test-strategy work below, not before it.

### WHAT THE FRESH REVIEW FOUND

A real review of 61 files does not come back empty, and this one raised **85 new findings**. Twenty-five are now fixed. The remaining **60 are left open on purpose** — listed, not swept.

This is worth stating plainly, because the earlier version of this report said "0 open items". Zero meant "nothing outstanding against a review conducted before any of this work existed." Eighty-five meant twenty reviewers looked at the code that actually ships today. The second number is the useful one, and 60 of it is still outstanding.

Nothing left open is a correctness bug. The eight that were are in the section above; what remains is structure, naming, contracts and consistency — the kind of finding that is real but whose cost is paid later, by the next person to change that file. Each is a refactor with regression risk, against screens whose test coverage is the thinnest part of the suite. Immediately before user testing, that is the wrong trade, so it was not made.

They are recorded rather than dismissed, so nothing has to be rediscovered:

```
desloppify show review --status open
```

The larger clusters are the filter panel's duplicated category handlers, an 88-line hand-written tag builder sitting directly above its own table-driven replacement, repeated grouping logic in the heat maps, and the detail drawer's over-wide prop list. None of it blocks testing what is on the branch now.

## THE ONE THING WORTH DOING NEXT

---

Test strategy is the lowest-scoring dimension at 76.0, and unlike everything else on the deferred list, it is not tidying. It is the gap most likely to let a real bug through.

Three things carry no real tests. The **SQLite layer** — every query behind every list in the app — is replaced by a stand-in everywhere it appears, so no test has ever run a real query. The **network client** is likewise mocked wholesale; the timeout fix described above is verified by reading it, not by a test. And the **backend's AI paths** are tested only with the key deliberately unset, which exercises the fallback and never the feature.

Also worth knowing: `npm test` runs only the app's suites. The backend's 70 tests — the only place a real database and a real HTTP server are exercised together — need a separate command, and there is no CI to insist on it. This is a piece of work with a genuine design decision at the front of it (a real database in tests, or a better fake?), which is why it is being raised here rather than started.

## NEEDS CHECKING ON A REAL DEVICE

---

Everything below passes in the test suite, but the suite cannot prove these particular things. They're worth ten minutes on a phone before shipping.

```
git fetch && git checkout krishnav8
npm install          # two packages were removed, so this matters
npm start
# the AI search, comparison and dictation features also need:
cd backend && npm install && npm run start
```

## First launch on a clean install — the one that matters

The app copies its bundled database into storage the first time it runs. That code moved to the current Expo file API, and **no test can verify it**: every test replaces the file system with a stand-in, so passing tests only prove the new code agrees with a stand-in I also wrote. If it's wrong, the app can't open its database at all — it fails at startup, not subtly.

Delete the app, install fresh, launch. If the catalogue loads, it works. The change is isolated in its own commit, so it's a clean revert if not.

### 02 Export a solution to a file

The other half of the same file-system change. Tap download on any solution and confirm the share sheet opens with a readable file.

### 03 Navigation in a release build

The app was disabling native navigation screens in every build to work around a problem that only affects Expo Go. That's now limited to Expo Go, so a release build gets native screens for the first time. Worth tapping through the tabs and back gestures once.

### 04 Text size set to Large

About 150 pieces of text that previously ignored this setting now follow it. That's the intended fix, but it means some screens will lay out differently at the large size than they ever have.

## DELIBERATELY LEFT ALONE

- **The OpenAI key is out of the app, but not yet rotated.** Nothing in the app reads it any more, and the unused `EXPO_PUBLIC_OPENAI_API_KEY` entry has been deleted — the same key already lives in `backend/.env`, where it belongs, so nothing was lost. What remains is that any build made while it was in the app still contains it, on whatever devices have those builds. Only rotating the key at OpenAI fixes that, and your standing instruction was not to, so I haven't.

- **The backend is not locked down yet.** It now supports requiring a client token, but that stays off until you set `API_CLIENT_TOKEN` on the server — turning it on without that would break builds already in users' hands. What changed this round is only that the setting is now *documented* on both sides, so turning it on is no longer a trap.
- **The other 60 review findings are open, not closed.** They were not marked as won't-fix, because they are deferred rather than rejected. The reasoning is in the section above.
- `assets/db/atiokb.db` **was never touched**, and I checked after every commit.
- **On branch `krishnav8`** — 39 commits. Each is self-contained with the reasoning in the message, so anything here can be reverted on its own. Nothing was merged to `main`.

---

strict 87.5 · mechanical 96.2 · 514 app tests · 70 backend    branch krishnav8 · pushed  
database asset unchanged